Lightweight RL Scheduling Prototype Implementation Guide

1 Purpose

This document is a concise but concrete implementation brief for coding agents assisting with a course project. The goal is to build a **lightweight prototype demo** for reinforcement-learning-based daily task scheduling in an ADHD-inspired toy environment.

This is **not** a clinically realistic system. It is a **toy benchmark** designed to demonstrate that an RL agent can outperform simple heuristics in a stochastic scheduling environment with fluctuating energy and focus.

2 Project Goal

Build a prototype with the following components:

- a custom Gymnasium environment for daily scheduling,
- a random task generator,
- three heuristic baselines,
- one RL agent (PPO via stable-baselines3),
- an evaluation script,
- a rollout/demo visualization for one simulated day.

The final deliverable should support a short course-project demo and a simple empirical comparison.

3 Hard Scope Constraints

Keep the project intentionally small.

3.1 Must include

- 16 time slots per day,
- 4 to 6 tasks per day,
- hidden/internal user variables: `energy`, `focus`,
- actions: choose one unfinished task or `REST`,

- one RL algorithm: PPO,
- three baselines:
  - random,
  - earliest-deadline-first (EDF),
  - energy-aware heuristic,
- evaluation on multiple simulated days,
- one human-readable sample-day demo visualization.

3.2 Should *not* include

- real user personalization,
- heavy dataset preprocessing pipelines,
- many RL algorithms,
- complex NLP over task text,
- clinical claims,
- overly large state/action spaces,
- complicated POMDP machinery for the first version.

For the prototype, it is acceptable to expose noisy energy/focus values directly in the observation.

4 Recommended Repository Structure

```
project_root/
  config.py
  task_generator.py
  env.py
  baselines.py
  train.py
  evaluate.py
  demo.py
  utils.py
  README.md
  report_figures/
  models/
```

5 Functional Requirements by File

5.1 `config.py`

Central location for all tunable constants.

Include parameters such as:

- `NUM_SLOTS = 16`,
- `MIN_TASKS = 4`,
- `MAX_TASKS = 6`,
- reward coefficients,
- energy/focus decay values,
- rest recovery values,
- crash probability,
- PPO training steps,
- random seeds.

Do not scatter magic constants across files.

5.2 `task_generator.py`

Generate a synthetic task set for one day.

Each task should contain:

- `id`,
- `difficulty` in $[0, 1]$,
- `priority` in $\{1, 2, 3\}$,
- `duration` in $\{1, 2, 3\}$ meaning required successful work units,
- `deadline` as an integer slot index in $[0, \texttt{NUM_SLOTS} - 1]$,
- `remaining` initialized to `duration`,
- `done` boolean.

Implement:

```
generate_task_set(num_tasks: int, num_slots: int, rng) -> list[dict]
```

Task sampling guidelines:

- include a mix of easy and hard tasks,
- include both early and late deadlines,
- include both short and long tasks,
- ensure at least one moderately urgent task appears often.

Do not use task text or NLP.

5.3 `env.py`

Implement a custom Gymnasium environment, e.g. `ADHDSchedulingEnv`.

Episode definition

- One episode = one day.
- Each day has `NUM_SLOTS = 16` slots.
- Episode ends when:
  - all slots are consumed, or
  - all tasks are completed.

Internal state

Keep these internal variables:

- current slot index,
- energy in $[0, 1]$,
- focus in $[0, 1]$,
- current consecutive work streak,
- list of tasks with remaining durations and done flags.

Observation

For the prototype, use a flat numeric vector containing:

- normalized current slot,
- energy,
- focus,
- normalized work streak,
- for each task (pad if needed up to max task count):
  - remaining duration,
  - difficulty,
  - priority,
  - normalized deadline,
  - done flag.

Use fixed-size padding to support PPO.

Action space

Discrete action space:

- action `0 ... N-1`: attempt one slot of work on task i,
- action `N`: `REST`,

where $N =$ `MAX_TASKS` and tasks beyond the current day's count should be treated as invalid/-padded.

Invalid task actions should be handled safely. Recommended options:

- either penalize invalid actions lightly and convert them to `REST`, or
- mask them if you later add action masking.

For the first version, the simplest approach is to penalize lightly and treat invalid actions as `REST`.

Transition logic

When the agent chooses a task:

- increase work streak,
- decrease energy,
- decrease focus,
- optionally apply a focus crash with small probability,
- compute progress probability based on energy, focus, and task difficulty,
- with that probability, decrease the task's remaining duration by 1,
- if remaining duration reaches 0, mark the task completed.

When the agent chooses `REST`:

- recover energy,
- recover focus,
- reduce or reset work streak.

Suggested simple formulas

These are toy dynamics, not scientifically calibrated.

If action is task i with difficulty d:

$$\texttt{energy} \leftarrow \mathrm{clip}(\texttt{energy} - 0.08 - 0.05d, 0, 1),$$
$$\texttt{focus} \leftarrow \mathrm{clip}(\texttt{focus} - 0.06 - 0.08d, 0, 1).$$

With probability p_{crash}, do

$$\texttt{focus} \leftarrow \mathrm{clip}(\texttt{focus} - 0.30, 0, 1).$$

Define success probability as

$$p_{\text{success}} = \sigma\big(3(\texttt{energy} + \texttt{focus} - d - 0.8)\big),$$

and with probability p_{success}, task progress increases by 1 unit.

If action is `REST`:

$$\begin{aligned} \texttt{energy} &\leftarrow \text{clip}(\texttt{energy} + 0.15, 0, 1), \\ \texttt{focus} &\leftarrow \text{clip}(\texttt{focus} + 0.18, 0, 1), \\ \texttt{work_streak} &\leftarrow 0. \end{aligned}$$

You may also optionally add a mild time-of-day drift to energy.

Reward

Keep reward simple and interpretable.

Suggested reward design:

- $+2 \times$ `priority` when a task is completed,
- -1 when a deadline passes for an unfinished task (apply only once per task),
- -0.5 if work streak exceeds a threshold such as 3 consecutive work slots,
- $+0.2$ for resting when energy is very low,
- small step penalty optional, but avoid making the reward too dense or complex.

Required methods

Implement standard Gymnasium methods:

- `__init__`,
- `reset`,
- `step`,
- `render` (optional textual render is enough).

Also useful helper methods:

- `_get_obs`,
- `_apply_task_action`,
- `_apply_rest_action`,
- `_check_deadlines`,
- `_all_tasks_done`.

5.4 `baselines.py`

Implement the following baseline policies.

Random policy

Chooses randomly among unfinished tasks and `REST`.

EDF policy

Picks the unfinished task with earliest deadline; if energy is extremely low, it can choose `REST`.

Energy-aware heuristic

Use simple logic:

- if energy or focus is below a threshold, choose `REST`,
- otherwise choose the unfinished task with highest priority among tasks whose difficulty is feasible.

A simple score can be:

$$\texttt{score} = 2 \cdot \texttt{priority} - \texttt{difficulty} - 0.1 \cdot \texttt{time_to_deadline}.$$

Implement a common interface such as:

```
select_action(obs, env) -> int
```

Policies should be deterministic except for the random policy.

5.5 `train.py`

Train PPO using stable-baselines3.

Requirements:

- create training environment,
- initialize PPO with MLP policy,
- train for a moderate number of timesteps,
- save model to `models/ppo_scheduler.zip`,
- optionally log training reward.

Suggested starting point:

```
PPO("MlpPolicy", env, verbose=1)
```

Train for around `20_000` to `100_000` timesteps depending on speed. Do not over-tune at the start. The first goal is simply to learn a policy that clearly beats random.

5.6 `evaluate.py`

Compare policies over many sampled days.

Policies to compare:

- random,
- EDF,
- energy-aware heuristic,
- trained PPO.

Run each policy on a reasonably sized evaluation set, such as 100 to 300 episodes.

Compute the following metrics:

- average total priority-weighted completed value,
- average number of completed tasks,
- urgent-task completion rate,
- average number of `REST` actions,
- average number of missed deadlines.

Produce:

- console summary table,
- bar plots for the main metrics.

Keep the evaluation simple and reproducible.

5.7 `demo.py`

Create a human-readable rollout demo.

Run all policies on the same sampled day and record:

- action taken at each slot,
- energy and focus over time,
- task progress and completion,
- whether deadlines were missed.

Output at least one visual summary such as:

- action timeline by slot,
- energy/focus line plot,
- task completion summary.

This file is important for the final presentation. A good demo is often more persuasive than raw training curves.

5.8 `utils.py`

Put shared helper functions here.

Examples:

- seeding utilities,
- sigmoid function,
- plotting helpers,
- rollout recording helpers,
- pretty-printing tasks.

6 Implementation Order

Coding agents should follow this order.

6.1 Phase 1 — environment skeleton

1. Implement `config.py`.
2. Implement `task_generator.py`.
3. Implement `env.py` with `reset`/`step`/`_get_obs`.
4. Test random stepping manually.

6.2 Phase 2 — baselines and sanity checks

5. Implement `baselines.py`.
6. Write a small loop to run baseline rollouts.
7. Verify behavior by printing trajectories.

6.3 Phase 3 — RL training

8. Implement `train.py`.
9. Train PPO.
10. Save the model.

6.4 Phase 4 — evaluation and demo

11. Implement `evaluate.py`.
12. Implement `demo.py`.
13. Generate 2 to 4 simple plots.

Do not jump into PPO before the environment is manually verified.

7 Acceptance Criteria

The prototype is considered successful if the following hold.

7.1 Minimum success criteria

- environment runs without crashing,
- random task sets are generated correctly,
- all three baselines run end-to-end,
- PPO trains and produces a saved model,
- evaluation compares all policies on multiple episodes,
- demo visualization shows at least one full-day rollout.

7.2 Desired empirical outcome

- PPO clearly beats random,
- PPO is competitive with or slightly better than EDF and the energy-aware heuristic on at least one main metric,
- learned behavior is interpretable, e.g. resting when exhausted or prioritizing urgent tasks.

8 Common Failure Modes to Avoid

- reward function too complicated,
- state representation too large,
- success probability too random to learn from,
- invalid actions causing training instability,
- no visual rollout inspection until the end,
- spending too much time trying multiple RL algorithms.

If PPO does not learn:

- reduce stochasticity,
- simplify reward,
- shorten task durations,
- make deadline structure clearer,
- verify that baseline heuristics behave sensibly.

9 Coding Style Requirements

- Use Python 3.
- Use `gymnasium` rather than old `gym` if possible.
- Use `stable-baselines3` for PPO.
- Keep functions short and modular.
- Add docstrings to major classes/functions.
- Use type hints where convenient.
- Prefer readability over premature optimization.
- Avoid unnecessary abstraction.

10 Suggested Prompting Strategy for Coding Agents

When asking a coding agent to implement pieces, do it module by module.

10.1 Good prompt example for `env.py`

> Implement `env.py` for a custom Gymnasium environment named `ADHDSchedulingEnv` using the specification in the implementation guide. Keep the environment lightweight and readable. Use a discrete action space for up to 6 tasks plus REST, fixed-size padded observations, simple toy energy/focus dynamics, and a clear reward function. Include docstrings and comments.

10.2 Good prompt example for `baselines.py`

> Implement `baselines.py` with three policies: random, earliest-deadline-first, and energy-aware heuristic. Use a shared interface `select_action(obs, env)` and keep the logic simple and deterministic except for the random policy.

10.3 Good prompt example for `evaluate.py`

> Implement `evaluate.py` to compare random, EDF, energy-aware, and trained PPO policies across many episodes. Report average total completed value, urgent completion rate, completed task count, missed deadlines, and rest count. Also create simple bar plots.

11 Minimal Demo Story for Presentation

The intended presentation message is:

> We built a toy daily scheduling environment with fluctuating energy and focus, compared several rule-based schedulers against PPO, and found that a learned policy can better balance difficult tasks, deadlines, and rest in a stochastic setting.

Do not claim clinical validity. Do not overstate realism.

12 Stretch Goals (Only If Time Permits)

Only after the full prototype works, consider:

- mild partial observability,
- action masking,
- recurrent policy,
- one extra ablation such as removing crashes,
- one extra visualization.

These are optional. The base prototype should remain lightweight.

13 Final Note to Coding Agents

Prioritize a complete end-to-end working prototype over realism. A small, stable, interpretable environment with sensible comparisons is much better than an ambitious but unfinished system.